



**UNIVERSIDAD NACIONAL DEL CENTRO DEL PERU
FACULTAD DE INGENIERIA DE SISTEMAS
DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS
PROGRAMA DE INGENIERÍA DE SISTEMAS**



Guía Práctica SEMANA 02

Asignatura: Desarrollo de Aplicaciones Web (IS093A)

Unidad I: Desarrollo Web Frontend

Tema: JavaScript & TypeScript, Manipulación del DOM, Funciones Avanzadas (Closures, IIFE, Arrow), Canvas API y Animaciones

Modalidad: Presencial / Laboratorio

APELLIDOS Y NOMBRES: Barja Ortiz Erick Gerson

OBJETIVO DE LA PRÁCTICA

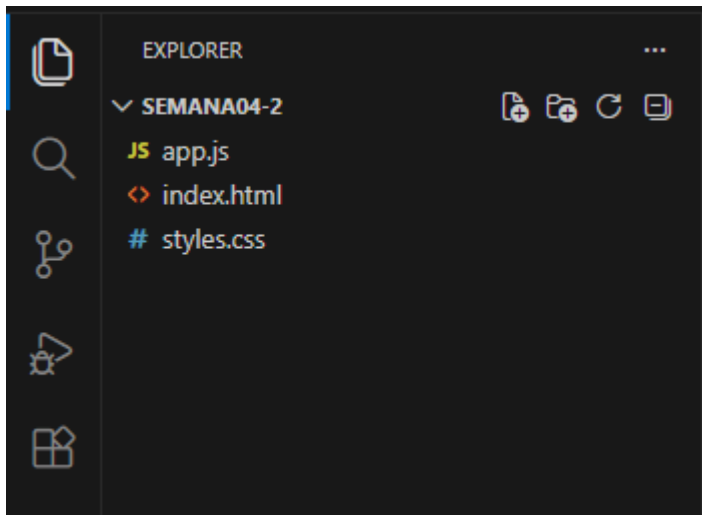
Manipular el Árbol DOM dinámicamente con JavaScript moderno (ES6+), aplicar estructuras de control, funciones flecha, autoinvocadas (IIFE) y closures para gestión de estado, e implementar una animación interactiva usando Canvas API. Comprender el motor de ejecución JS, el Event Loop, el ciclo de renderizado y estrategias de optimización (requestAnimationFrame, prevención de fugas de memoria).

RECURSOS REQUERIDOS

- Visual Studio Code (Extensiones: Live Server, JavaScript/TypeScript Language Features, ESLint, Canvas Preview)
- Navegador Chrome/Firefox + DevTools (Consola, Elements, Performance, Memory)
- TypeScript (opcional: tsc global o playground online para validación de tipos)
- Documentación oficial: MDN Web Docs (DOM, Canvas, requestAnimationFrame)
- Conexión a internet

PARTE 1: EJERCICIO PASO A PASO

1. Crear index.html, app.js (o app.ts), estructurar UI con controles y <canvas>



```
index.html > html > body > header.site-header > div.header-inner
1  <!DOCTYPE html>
2  <html lang="es">
3  <head>
4      <meta charset="UTF-8" />
5      <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6      <meta name="description" content="Página de prueba para una sucursal en El Tambo del hospital
7      MedVita">
8      <title>MedVita – Sucursal Centro</title>
9      <link rel="stylesheet" href="styles.css" />
10     <!--
11         Google Fonts:
12         Esto se referencia solo para el uso de fuentes de nuestra pagina
13         Se usa Rajdhani para títulos, Lato para texto normal
14     -->
15     <link rel="preconnect" href="https://fonts.googleapis.com" />
16     <link
17         href="https://fonts.googleapis.com/css2?family=Rajdhani:wght@500;600;700&family=Lato:wght@300;400;
18         700&display=swap"
19         rel="stylesheet"
20     />
21 </head>
```

```
192
193     <!-- ===== FOOTER ===== -->
194     <footer class="site-footer">
195         <p>&copy; 2025 MedVita Sucursal Centro · Todos los derechos reservados</p>
196         <p class="footer-sub">Dispuestos a contribuir a la buena salud</p>
197     </footer>
198
199     <!-- Cargamos el script al final para que el DOM ya esté listo -->
200     <script src="app.js" defer></script>
201
```

2. Implementar IIFE para aislamiento de scope, closures para estado de animación y arrow functions para handlers

```
html # styles.css JS app.js X
s > <function>
(function () {
  "use strict"; // Esta en Modo estricto para ayudar a detectar errores comunes
```

```
<> index.html # styles.css JS app.js X
JS app.js > <function> > crearContadorFPS
19 (function () {
30   function crearContadorFPS() {
31     // Variables privadas que solo son accesibles dentro de la función
32     var frames = 0;
33     var tiempoAnterior = performance.now();
34     var fps = 0;
35
36     return {
37       /* Se llama en cada frame del canvas (esto es del paso 4) */
38       actualizar: function (tiempoActual) {
39         frames++;
40         var diferencia = tiempoActual - tiempoAnterior;
41
42         // Cada segundo calculamos los FPS reales
43         if (diferencia >= 1000) {
44           fps = Math.round((frames * 1000) / diferencia);
45           frames = 0;
46           tiempoAnterior = tiempoActual;
47         }
48         return fps;
49       },
50       obtener: function () {
51         return fps;
52       }
53     };
54   }
55 }
```

```

56  /* -----
57  * PASO 2 (Mejorado por IA): CLOSURE para estado de partículas
58  * crearEstadoAnimacion devuelve funciones que leen/escriben
59  * variables privadas. El array "particulas" y "activa" viven
60  * dentro del closure y no se pueden tocar desde afuera.
61  * ----- */
62  function crearEstadoAnimacion() {
63      var particulas = []; // array interno – no accesible desde fuera
64      var activa = false;
65      var idFrame = null;
66
67      return {
68          getParticulas: function () { return particulas; },
69          setParticulas: function (arr) { particulas = arr; },
70          estaActiva: function () { return activa; },
71          activar: function () { activa = true; },
72          getIdFrame: function () { return idFrame; },
73          setIdFrame: function (id) { idFrame = id; }
74      };
75  }
76

```

```

107 // Arrow function como callback
108 var observarSecciones = (function () {
109     var obs = new IntersectionObserver(
110         function (entries) {
111             entries.forEach(function (entry) {
112                 if (entry.isIntersecting) {
113                     // Quitamos "active" de todos y lo ponemos en el que corresponde
114                     navLinks.forEach(function (link) {
115                         link.classList.remove("active");
116                     });
117                     var linkActivo = document.querySelector(
118                         '.nav-link[href="#' + entry.target.id + '"]'
119                     );
120                     if (linkActivo) linkActivo.classList.add("active");
121                 }
122             });
123         },
124         { threshold: 0.4 } // sección activa cuando el 40% es visible
125     );
126
127     secciones.forEach(function (sec) { obs.observe(sec); });
128 })();

```

3. Manipulación DOM: `querySelector`, `addEventListener`, modificación dinámica de estilos/clases, validación de inputs

```

93     if (menuBtn && mainNav) {
94         menuBtn.addEventListener("click", function () {
95             mainNav.classList.toggle("open"); // Para que el menu hamburguesa funcione (es responsive)
96         });
97     }
98

```

```

130     /* -----
131     * PASO 3 (IA usada para las descripciones): Generar tarjetas de servicios con manipulación DOM
132     * En lugar de escribirlas a mano en HTML, las creamos con JS
133     * para practicar createElement e innerHTML.
134     * ----- */
135     var SERVICIOS = [
136         { icono: "🫀", nombre: "Cardiología", desc: "Evaluación y seguimiento del corazón y sistema circulatorio." },
137         { icono: "🧠", nombre: "Neurología", desc: "Diagnóstico de enfermedades del sistema nervioso." },
138         { icono: "🦷", nombre: "Odontología", desc: "Cuidado dental preventivo y correctivo para toda la familia." },
139         { icono: "👶", nombre: "Pediatria", desc: "Atención especializada para niños desde recién nacidos." },
140         { icono: "🦴", nombre: "Traumatología", desc: "Fracturas, lesiones musculoesqueléticas y rehabilitación." },
141         { icono: "👁", nombre: "Oftalmología", desc: "Revisión de visión, cirugía y tratamiento ocular." },
142         { icono: "🩺", nombre: "Medicina General", desc: "Consultas de rutina, diagnósticos y referencias." },
143         { icono: "👩", nombre: "Ginecología", desc: "Salud femenina, embarazo y controles perinatales." }
144     ];
145

```

```

171
172     // Arrow function para limpiar el error de un campo
173     var limpiarError = function (inputId, errorId) {
174         var campo = document.querySelector("#" + inputId);
175         var error = document.querySelector("#" + errorId);
176         if (campo) campo.classList.remove("input-error");
177         if (error) error.textContent = "";
178     };
179
180     // Arrow function para marcar un campo como inválido
181     var mostrarError = function (inputId, errorId, mensaje) {
182         var campo = document.querySelector("#" + inputId);
183         var error = document.querySelector("#" + errorId);
184         if (campo) campo.classList.add("input-error");
185         if (error) error.textContent = mensaje;
186     };
187

```

4. Canvas: contexto 2D, requestAnimationFrame, loop de renderizado, dibujo básico (arc, fillRect, stroke)

```

235  /* -----
236  * PASO 4 (IA usada para mejorar la funcion de crear particulas):
237  * Canvas 2D con requestAnimationFrame
238  * Dibujamos partículas que flotan en el fondo del hero.
239  * Usamos delta time (dt) para que la velocidad sea igual
240  * sin importar cuántos FPS tenga el dispositivo del usuario.
241  * ----- */
242  var contexto = canvas ? canvas.getContext("2d") : null;
243  var estado = crearEstadoAnimacion(); // closure del estado
244  var contFPS = crearContadorFPS(); // closure del FPS
245
246  /* Ajusta el tamaño del canvas al de la ventana */
247  function ajustarCanvas() {
248    if (!canvas) return;
249    canvas.width = canvas.offsetWidth;
250    canvas.height = canvas.offsetHeight;
251  }
252
253  /* Crea una partícula con valores aleatorios */
254  function crearParticula() {
255    return {
256      x: Math.random() * canvas.width,
257      y: Math.random() * canvas.height,
258      r: Math.random() * 2 + 0.5, // radio entre 0.5 y 2.5
259      vx: (Math.random() - 0.5) * 30, // velocidad horizontal
260      vy: (Math.random() - 0.5) * 20, // velocidad vertical
261      op: Math.random() * 0.5 + 0.1 // opacidad entre 0.1 y 0.6
262    };
263  }
264

```

```

265  /* Inicializa el array de partículas */
266  function iniciarParticulas(cantidad) {
267    var arr = [];
268    for (var i = 0; i < cantidad; i++) {
269      arr.push(crearParticula());
270    }
271    estado.setParticulas(arr);
272  }
273

```

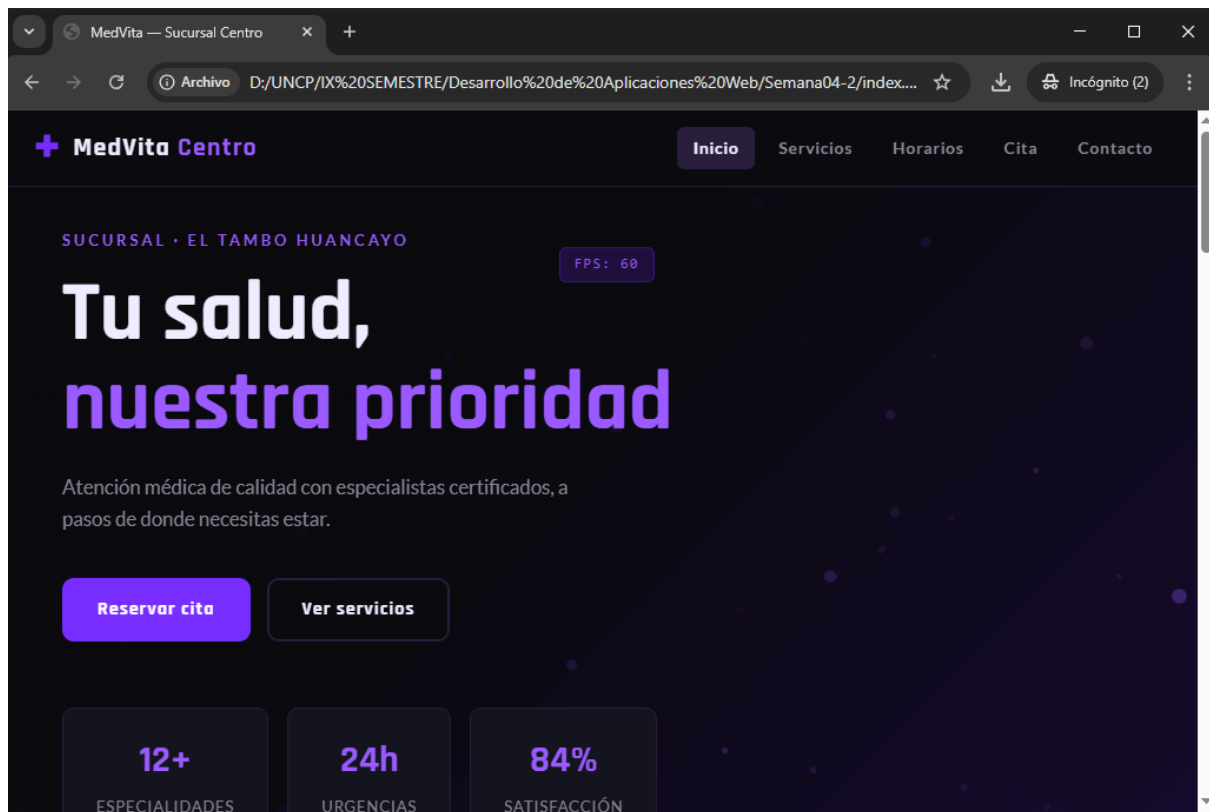
5. Depuración y optimización: Console Profiler, Memory Snapshot, detección de listeners huérfanos, FPS counter

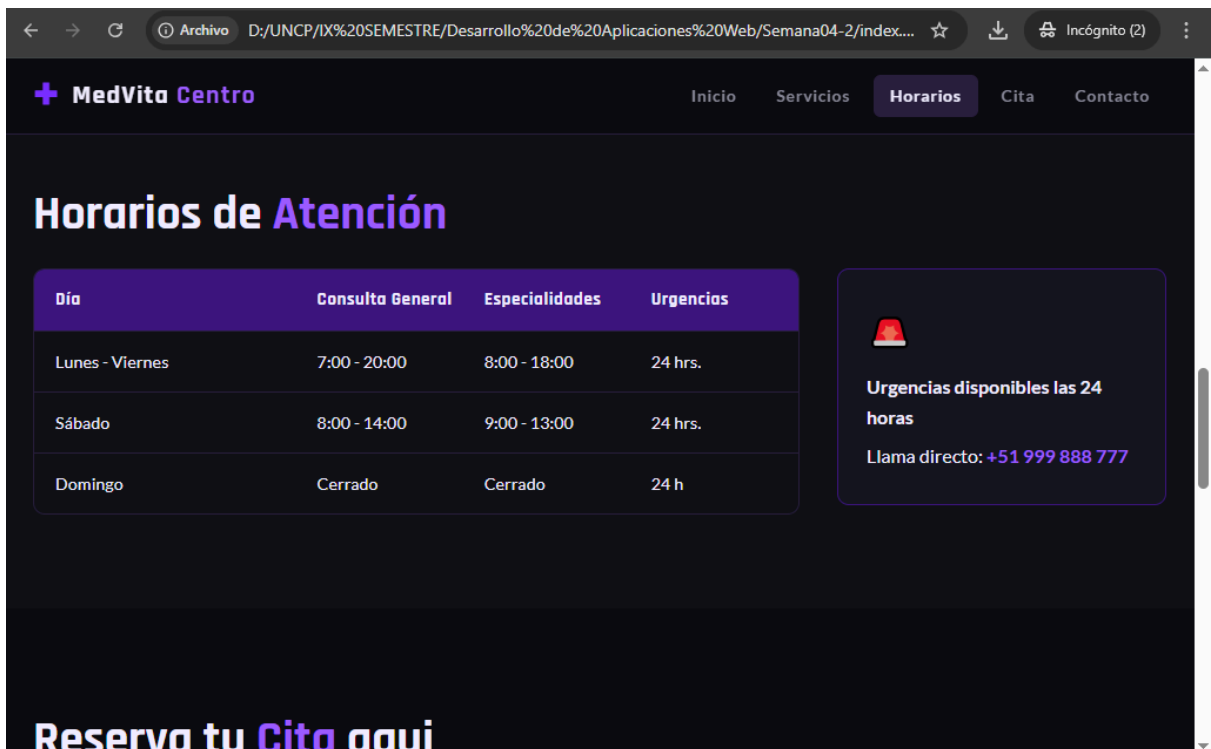
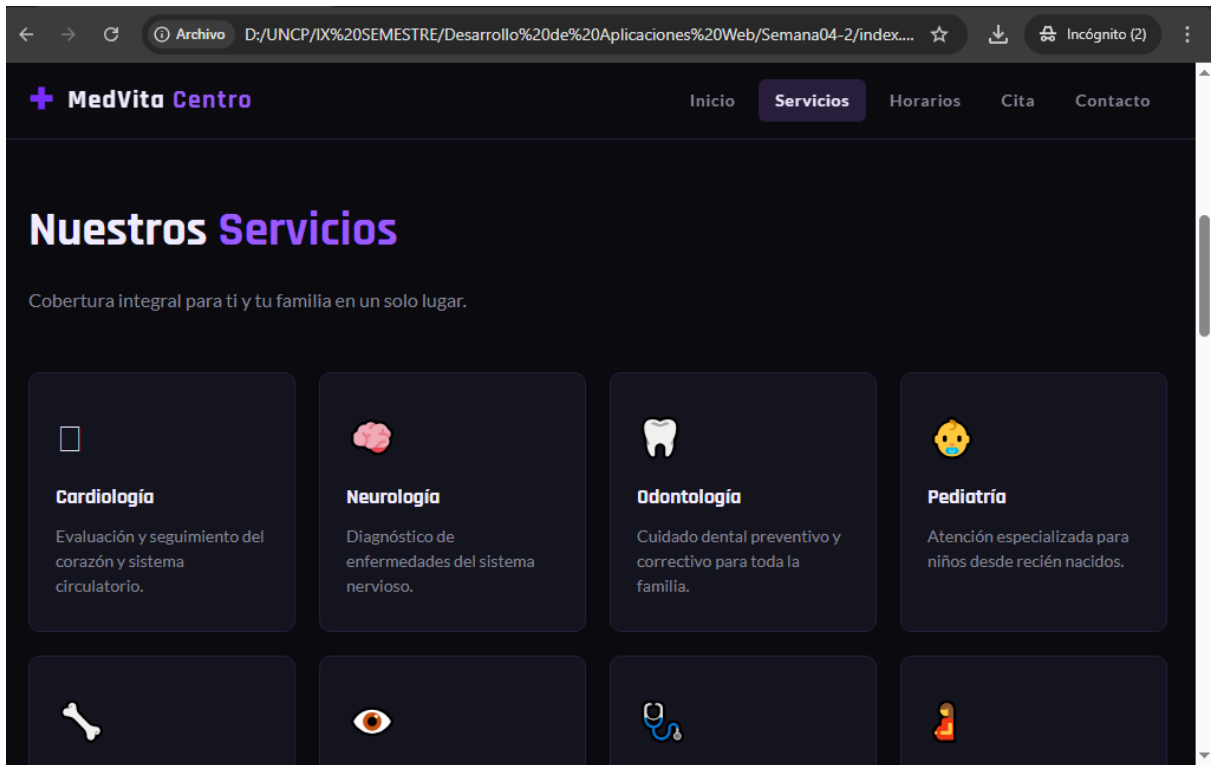
```

367 window.addEventListener("load", function () {
368     var nodosTotales = document.querySelectorAll("*").length;
369     console.log("[MedVita] Página cargada.");
370     console.log("[MedVita] Nodos DOM totales:", nodosTotales);
371     console.log("[MedVita] Servicios generados:", SERVICIOS.length);
372
373     // Iniciamos el canvas luego de que la página cargó
374     iniciarAnimacion();
375 });
376
377 /*

```

6. PAGINA WEB IMPLEMENTADA LOCALMENTE





← → ↻ Archivo D:/UNCP/IX%20SEMESTRE/Desarrollo%20de%20Aplicaciones%20Web/Semana04-2/i... 🔍 ☆ ⬇️ Incógnito (2) ⋮

+ MedVita Centro Inicio Servicios Horarios Cita **Contacto**

Reserva tu Cita aquí

Completa el formulario y nos pondremos en contacto

NOMBRE COMPLETO

Ej. Luis Hernández López

CORREO ELECTRÓNICO

ejemplo@gmail.com

ESPECIALIDAD

— Seleccionar una Esp. — ▾

FECHA PREFERIDA

dd/mm/aaaa 📅

Solicitar cita

← → ↻ Archivo D:/UNCP/IX%20SEMESTRE/Desarrollo%20de%20Aplicaciones%20Web/Semana04-2/i... 🔍 ☆ ⬇️ Incógnito (2) ⋮

+ MedVita Centro Inicio Servicios Horarios Cita **Contacto**

dd/mm/aaaa 📅

Solicitar cita

¿Dónde encontrarnos?

📍 Jr. Ucayali N° 850, El Tambo Huancayo

☎️ +51 999 888 7777

✉️ atencion.cliente@medvita.pe

🕒 Urgencias 24 h · L-V 7:00 - 20:00

📍 El Tambo Huancayo

Jr. Ucayali N° 850

© 2025 MedVita Sucursal Centro · Todos los derechos reservados

Dispuestos a contribuir a la buena salud

Link en GitHub: <https://github.com/GErickOBTS/Semana04-2>

